Reproducing and Extending the MTTD Dynamic Model-Selection Framework: Five ML Models on a 2023 Cricket World Cup Workload with CloudSim Evaluation

Abhinav Mishra, Garv Gujral, and Shashank Yadav

Abstract

Nashaat *et al.* (2026) proposed MTTD, a Monitor–Train–Test–Deploy framework that trains several workload predictors, deploys the one with the lowest error score, and retrains when an adaptive threshold is breached on two consecutive cycles. They evaluated three models (ARIMA, SVR, Random Forest) on FIFA World Cup server logs. We reproduce the framework with five models, adding XGBoost and a PyTorch LSTM, and apply it to a new hourly workload derived from daily Wikipedia page views (user traffic) of the 2023 ICC Cricket World Cup. The predictions are then replayed in a CloudSim provisioning simulation. On four synthetic workload patterns, ARIMA scored best in every case. On the cricket trace, the test phase selected Random Forest (Score 6.49%); the best-scoring model then varied from cycle to cycle during 45 daily deployment cycles (Random Forest in 16, SVR in 11, XGBoost in 10, ARIMA in 8), and one retraining event at hour 552 re-selected Random Forest. Retraining reduced the deployment score from 52.43% to 32.78% compared with the same model without retraining (RMSE -28.7%, MAE -47.8%), and in CloudSim it used 17.1% fewer VM-hours with 15.2% fewer SLA-violation hours. Over the full deployment, SVR would have scored 22.41% in hindsight. When the model was re-selected more often than the two-strike rule allows, the active model changed 26 times (retraining and reselection every day) and 60 times (reselection every hour on the last 24 hours), and the score fell to 16.13% and 15.42%, below the best single model.

Index Terms

Cloud autoscaling, workload prediction, dynamic model selection, concept drift, flash crowds, CloudSim, XGBoost, LSTM.

I. Introduction

ELASTIC cloud platforms promise capacity that tracks demand, but the controllers that deliver it are mostly reactive. A threshold autoscaler, such as the Kubernetes Horizontal Pod Autoscaler [34], adds replicas only after utilisation has already crossed a target, so the first minutes of a surge are served by an undersized fleet [11], [12], [31]. Production systems at hyperscale therefore combine reactive control with forecasting [32], [33].

Live sports events are a hard case for forecasting. Audience attention concentrates in match windows, swings sharply between fixtures of different importance, and peaks during knockout games and finals. Such flash crowds are well documented for web workloads since the 1998 FIFA World Cup [13], [14]. A forecaster trained on the group stage sees a different distribution from the one it must serve during the semi-finals, an instance of concept drift [35].

Comparative studies repeatedly find that no single model dominates across workload patterns [3], [19], [25], [26]. Nashaat *et al.* [1] responded with MTTD, a Monitor–Train–Test–Deploy loop that trains several candidates, deploys the best, and retrains when an error-amplified threshold is breached on two consecutive cycles. Their evaluation used the 1998 and 2018 FIFA World Cup server logs and three candidates (ARIMA, SVR, Random Forest), and stopped at prediction accuracy.

This paper reproduces MTTD and extends it. Its four contributions are:

The authors are with Netaji Subhas University of Technology, New Delhi 110075, India.

Code, workload generator and simulation: https://github.com/garv-codes/ML-workload-prediction.

1) **Reproduction of the MTTD pipeline.** We implement all four phases of [1], including its performance score and two-strike adaptive retraining threshold, and repeat its synthetic-workload experiments.
2) **Five candidate models.** We add gradient boosting (XGBoost [8]) and a recurrent network (LSTM [9]) to the original ARIMA, SVR and Random Forest candidates.
3) **A 2023 Cricket World Cup workload.** In place of the FIFA logs, we disaggregate 56 daily English-Wikipedia page-view counts (user traffic) for the tournament article into 1,344 hourly intervals.
4) **CloudSim evaluation.** A bridge script exports MTTD's per-hour forecasts, which drive a trace-driven CloudSim [2] simulation of five provisioning policies: REACTIVE, MTTD without retraining, MTTD with retraining, ORACLE, and MTTD-Hybrid.
5) **Effect of reselection frequency.** We show how the active model changes when MTTD re-selects every day or every hour instead of only after two consecutive threshold breaches.

Section II reviews related work. Section III presents the architecture, formulation and data pipeline. Section IV reports the experimental setup and results. Section V concludes.

II. Related Work

Workload prediction for autoscaling has been surveyed extensively [11], [12], [28], [29]. We group prior work into four families, summarised in Fig. 1 and Table I.

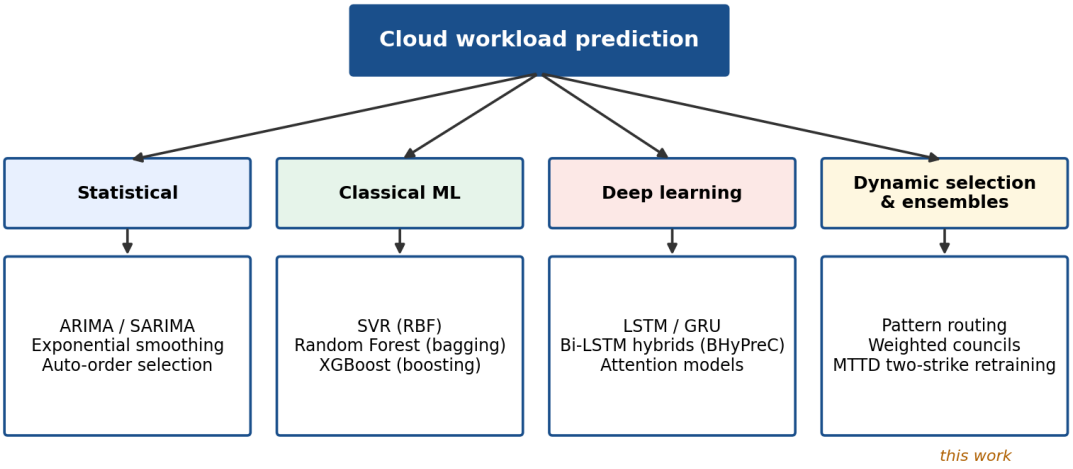


Fig. 1. Taxonomy of cloud workload prediction approaches. This work extends the dynamic-selection family.

A. Statistical Models

ARIMA models a series through autoregressive, differencing and moving-average terms [4]. Calheiros *et al.* [3] used ARIMA to forecast web requests and evaluated its effect on QoS in CloudSim. Automatic order selection [5] removes manual ACF/PACF tuning. Early predictive scalers such as PRESS [17] and CloudScale [18] paired signal-processing or Markov predictors with corrective control. ARIMA assumes linear, locally stationary dynamics, so strong intraday seasonality degrades it unless a seasonal term is modelled.

B. Classical Machine Learning

Support vector regression [6] and Random Forests [7] recast forecasting as supervised regression on a sliding window of lags. Empirical comparisons [16], [19] evaluated such learners against time-series models, and Kim *et al.* [19] found that no single predictor is best for all workload patterns. Later studies applied these learners directly to elastic provisioning [23], [30]. Gradient boosting with XGBoost [8] adds shrinkage and regularised tree growth.

C. Deep Learning

LSTMs [9] capture long-range temporal dependence through gated memory. Kumar and Singh [21], [22] trained neural forecasters with evolutionary and self-directed learning. Bi *et al.* [24] combined bidirectional and grid LSTMs (BG-LSTM). BHyPreC [10] stacks Bi-LSTM, LSTM and GRU layers for VM CPU prediction.

D. Dynamic Selection and Ensembles

Because no single model dominates, several works select or combine models online. Liu *et al.* [25] discriminate workload patterns and route each to a matching predictor. AME-WPC [26] combines several forecasting methods, and Messias *et al.* [27] combine time-series models with a genetic algorithm. CloudInsight [20] maintains a council of predictors and reweights them over time. MTTD [1] instead deploys one active model and retrains all candidates only when an error-amplified threshold is breached twice in a row.

TABLE I
REPRESENTATIVE WORKLOAD-PREDICTION APPROACHES

Ref.	Approach	Family	Dynamic selection	Evaluation scope
[3]	ARIMA	Statistical	No	Forecast accuracy and QoS impact in CloudSim
[15]	ARMA + look-ahead control	Statistical	No	Forecasting with autoscaling control
[16]	NN, linear regression	Classical ML	No	Forecast accuracy
[19]	21 predictors compared	Mixed	No	24 synthetic workloads; cost and deadline-miss simulation
[21]	NN + adaptive DE	Deep / evolutionary	No	Forecast accuracy
[10]	Bi-LSTM + LSTM + GRU (BHyPreC)	Deep learning	No	VM CPU forecast accuracy
[25]	Pattern-routed predictors	Dynamic	Yes (by pattern)	Forecast accuracy vs. ARIMA, SVM, LR
[20]	Predictor council (CloudInsight)	Ensemble	Yes (reweighting)	Real traces; trace-based provisioning simulation
[1]	MTTD: ARIMA, SVR, RF	Dynamic	Yes (two-strike)	Forecast accuracy on FIFA 1998 and 2018 logs
This work	MTTD: + XGBoost, LSTM	Dynamic	Yes (two-strike)	Forecast accuracy and CloudSim provisioning (5 policies)

This work follows [1] and adds two models, a new workload and a provisioning evaluation (last row of Table I).

III. PROPOSED METHODOLOGY AND SYSTEM ARCHITECTURE

A. Decoupled Two-Stage Architecture

The system separates prediction from provisioning (Fig. 2). Stage 1, in Python, runs the online MTTD loop over the hourly trace and records, for every deployment hour, the observed load, the active model's one-step-ahead forecast, the active model's name and any retraining event. The bridge script `mttd_bridge.py` writes these records as `predictions_retrain.csv` and, with retraining disabled, `predictions_noretrain.csv`. Stage 2, in Java, reads both files and replays each hour in CloudSim under five provisioning policies. Provisioning decisions do not feed back into the workload.

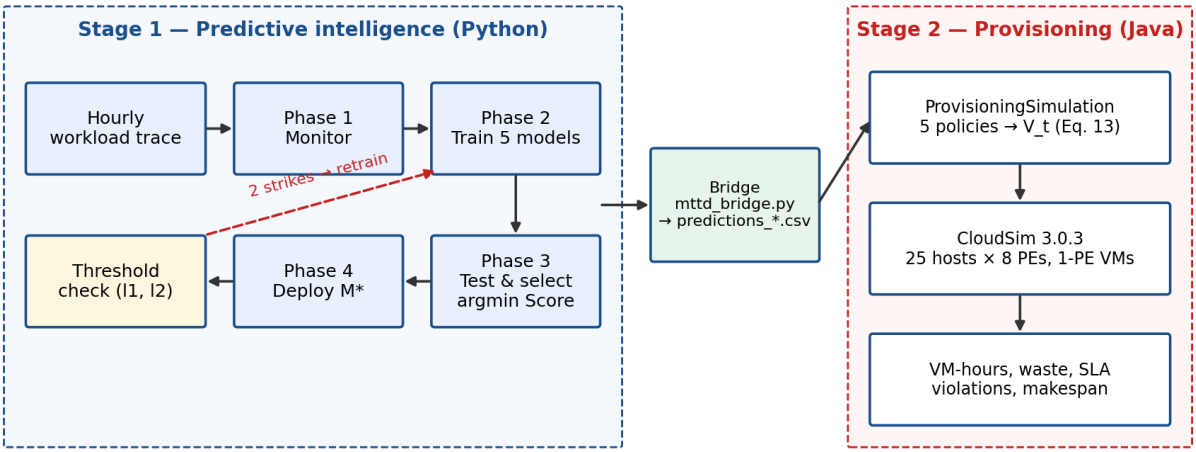


Fig. 2. Decoupled two-stage MTTD architecture: the Python forecasting loop (Stage 1), the CSV bridge, and the Java CloudSim provisioning simulation (Stage 2).

B. Problem Formulation and Metrics

Let y_t be the number of requests in hour t and $\hat{y}_t^{(i)}$ the one-step-ahead forecast of candidate $i \in \mathcal{M} = \{\text{ARIMA}, \text{SVR}, \text{RF}, \text{XGB}, \text{LSTM}\}$, made from observations up to $t-1$. Over an evaluation window of N hours we use the error measures of [1], [36]:

$$\text{RMSE}_i = \sqrt{\frac{1}{N}\sum_{t=1}^{N}\left(y_t - \hat{y}_t^{(i)}\right)^2}, \tag{1}$$

$$\text{MAE}_i = \frac{1}{N}\sum_{t=1}^{N}\left|y_t - \hat{y}_t^{(i)}\right|. \tag{2}$$

MTTD ranks models by the normalised, weighted score of [1] (its Eq. 4) with $w_1 = w_2 = 0.5$:

$$\text{Score}_i = 100\left(w_1\frac{\text{RMSE}_i}{\sqrt{\frac{1}{N}\sum_t y_t^2}} + w_2\frac{\text{MAE}_i}{\frac{1}{N}\sum_t y_t}\right). \tag{3}$$

The active model is the candidate with the lowest score on the test window:

$$M^* = \arg\min_{i\in\mathcal{M}} \text{Score}_i. \tag{4}$$

On the synthetic benchmarks we also report $\text{MAPE}_i = \frac{100}{N}\sum_t |y_t - \hat{y}_t^{(i)}|/|y_t|$.

C. Error-Amplified Adaptive Retraining Threshold

The active model is deployed for cycles of $n = 24$ hours. Let S_t be its score on cycle t. Following [1], the threshold for the next cycle is set from the latest healthy score with weight l_1, and set with weight l_2 after a first breach:

$$\text{Threshold}_{t+1} = S_t + l_1 S_t, \qquad l_1 = 0.75, \tag{5}$$

$$\text{Threshold}_{t+2} = S_{t+1} + l_2 S_{t+1}, \qquad l_2 = 0.50. \tag{6}$$

A strike counter k implements the two-strike rule, retraining at t if and only if $k_t = 2$:

$$k_t = \begin{cases} 0, & S_t \le \text{Threshold}_t, \\ k_{t-1} + 1, & S_t > \text{Threshold}_t. \end{cases} \tag{7}$$

Because each new threshold is derived from the score that produced it, a single bad cycle never triggers retraining; the error must escalate across two consecutive cycles.

D. The Four-Phase MTTD Loop

Algorithm 1 gives the online loop and Fig. 3 the same control flow. All five candidates are retrained and forecast every cycle so that per-model scores can be logged, but only the active model's forecasts drive provisioning.

Algorithm 1 Online MTTD with two-strike retraining

Require: series y, candidates $\mathcal{M}$, T_{tr}=240 h, T_{te}=24 h, cycle n=24 h, window w=24, l_1=0.75, l_2=0.50

$t \leftarrow T_{tr} + T_{te}$ ▷ Phase 1: monitor
fit every $m \in \mathcal{M}$ on $y[t{-}T_{tr}{-}T_{te} : t{-}T_{te}]$ ▷ Phase 2
$S_m \leftarrow$ Score of rolling forecast of $y[t{-}T_{te} : t]$ for all m
$M^* \leftarrow \arg\min_m S_m$; $\theta \leftarrow S_{M^*}(1{+}l_1)$; $k \leftarrow 0$ ▷ Phase 3
while $t < |y|$ **do** ▷ Phase 4: deploy
    forecast $y[t : t{+}n]$ one step ahead with M^*, observing truth
    $S \leftarrow$ Score of this cycle
    if $S \leq \theta$ **then**
        $k \leftarrow 0$; $\theta \leftarrow S(1{+}l_1)$
    else
        $k \leftarrow k + 1$
        if $k = 1$ **then**
            $\theta \leftarrow S(1{+}l_2)$
        else
            $k \leftarrow 0$; $t' \leftarrow t + n$
            refit all m on $y[t'{-}T_{tr}{-}T_{te} : t'{-}T_{te}]$
            re-test on $y[t'{-}T_{te} : t']$; $M^* \leftarrow \arg\min$; $\theta \leftarrow S_{M^*}(1{+}l_1)$
    $t \leftarrow t + n$

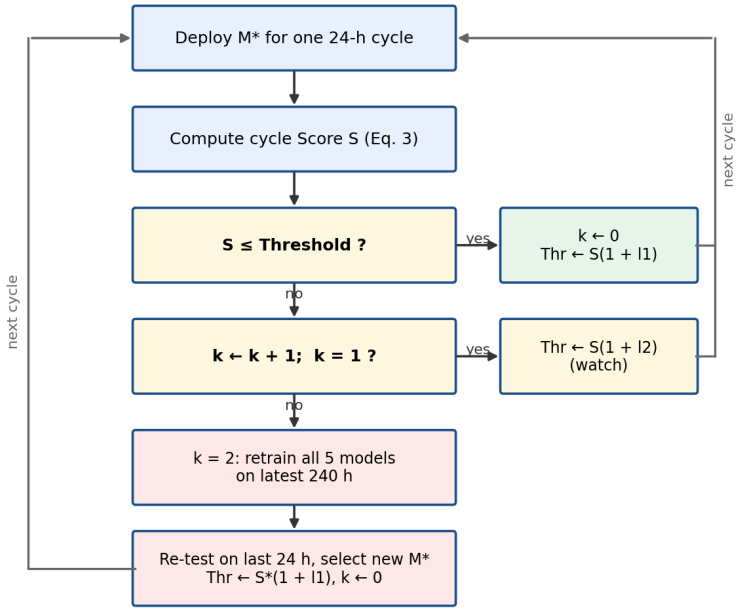


Fig. 3. Retraining and model-reselection flowchart implementing Eqs. (5)–(7).

E. Data Engineering: Daily-to-Hourly Disaggregation

Daily page views D_d for the English Wikipedia article "2023 Cricket World Cup" were exported from the Wikimedia Pageviews tool [39] for 1 Oct–25 Nov 2023 (56 days) with agent = user, which excludes bots and

automated traffic. Let $h \in \{0, \dots, 23\}$ be the hour of day. The match window $\mathcal{P} = \{14, \dots, 21\}$ receives 70% of the day's traffic and the 16 other hours share 30%:

$$b_{d,h} = \begin{cases} 0.70\, D_d/8, & h \in \mathcal{P}, \\ 0.30\, D_d/16, & h \notin \mathcal{P}. \end{cases} \tag{8}$$

Each hour is perturbed by bounded uniform noise:

$$\tilde{y}_{d,h} = b_{d,h}\, \varepsilon_{d,h}, \qquad \varepsilon_{d,h} \sim \mathcal{U}(0.92,\, 1.08). \tag{9}$$

The noisy hours are renormalised so that each day keeps its real total, and rounded with the residual assigned to the largest hour:

$$y_{d,h} = \operatorname{round}\!\left(\tilde{y}_{d,h}\, \frac{D_d}{\sum_{h'} \tilde{y}_{d,h'}} \right), \qquad \sum_h y_{d,h} = D_d. \tag{10}$$

A fixed random seed (42) makes the trace reproducible. Eq. (8) places 8.75% of daily traffic in each match hour and 1.875% in each other hour, so the trace steps up at 14:00 and down at 22:00 every day.

F. CloudSim Provisioning Model

Each VM serves $C = 4{,}000$ requests per hour: with 1,000 MIPS per VM, each request costs 900 MI. Requests are batched into cloudlets of 200 requests (180,000 MI, 180 s on one VM). For policy p with load estimate $\hat{L}_t^{(p)}$, the five policies use

$$\begin{aligned} \hat{L}_t^{\mathrm{REA}} &= y_{t-1}, & \hat{L}_t^{\mathrm{NR}} &= \hat{y}_t^{\mathrm{static}}, & \hat{L}_t^{\mathrm{OR}} &= y_t, \\ \hat{L}_t^{\mathrm{RT}} &= \hat{y}_t^{\mathrm{MTTD}}, & \hat{L}_t^{\mathrm{HY}} &= \max\!\big(\hat{y}_t^{\mathrm{MTTD}}, y_{t-1}\big). \end{aligned} \tag{11}$$

The required fleet uses the true load without headroom:

$$V_t^{\mathrm{req}} = \min\!\big(V_{\max}, \max(1, \lceil y_t/C \rceil)\big). \tag{12}$$

The provisioned fleet applies headroom $\eta = 1.10$ to the estimate, capped at $V_{\max} = 150$:

$$V_t^{(p)} = \min\!\big(V_{\max}, \max(1, \lceil \eta\, \hat{L}_t^{(p)}/C \rceil)\big). \tag{13}$$

CloudSim executes the hour's $K_t = \lceil y_t/200 \rceil$ cloudlets on the provisioned time-shared VMs, assigned round-robin by the broker. The hour violates the SLA if the makespan exceeds one hour:

$$\mathrm{SLA}_t^{(p)} = \mathbb{1}\big[T_t^{(p)} > 3600\,\mathrm{s}\big], \quad T_t^{(p)} \approx \big\lceil K_t/V_t^{(p)} \big\rceil \times 180\,\mathrm{s}. \tag{14}$$

We report total VM-hours, wasted VM-hours ($\sum_t \max(0, V_t^{(p)} - V_t^{\mathrm{req}})$), SLA-violation hours, under-provisioned hours ($V_t^{(p)} < V_t^{\mathrm{req}}$) and mean makespan. By Eq. (14), every under-provisioned hour is also an SLA violation, since some VM then holds at least 21 cloudlets (3,780 s).

IV. Experimental Setup and Results

A. CloudSim Datacenter Configuration

Table II lists the simulation parameters. The largest hourly load in the deployment window is 117,612 requests, which needs 30 VMs, or 33 with headroom, so the 150-VM cap never binds and ORACLE incurs no SLA violations.

TABLE II
SIMULATION PARAMETERS (PROVISIONINGSIMULATION.JAVA, CLOUDSIM 3.0.3)

Parameter	Value
Hosts	25 × (8 PEs × 1,000 MIPS, 16 GB RAM, time-shared VM scheduler)
VM	1 PE, 1,000 MIPS, 512 MB RAM, time-shared cloudlet scheduler
Per-VM capacity C	4,000 requests/h (900 MI per request)
Cloudlet size	200 requests = 180,000 MI (180 s on one VM)
Headroom η	1.10
Fleet cap $V_{\max}$	150 VMs (600,000 requests/h)
SLA	All cloudlets of the hour finish within 3,600 s
Horizon	1,080 hours (hours 264–1343), 5 policies

B. Candidate Models and Hyperparameters

All window-based models use the last 24 hourly values as features. The training window is 240 hours (10 days) and the test window 24 hours, matching the hourly FIFA-2018 configuration of [1]. Grid searches use 3-fold `GridSearchCV` with negative MSE [37]; the LSTM uses PyTorch [38].

TABLE III
CANDIDATE PREDICTORS ON THE CRICKET TRACE (`MTTD_BRIDGE.PY`)

Model	Configuration and search grid
ARIMA	`pmdarima.auto_arima` [5], non-seasonal, $p, q \leq 5$, d by unit-root test, stepwise; updated with each new observation
SVR	RBF; $C \in \{1, 10, 100\}$, $\epsilon \in \{0.001, 0.01\}$, $\gamma \in \{0.01, 0.1\}$; inputs scaled by the training maximum
Random Forest	100 trees; max depth $\in \{10, 40\}$; min samples/leaf $\in \{1, 3\}$
XGBoost	trees $\in \{100, 200\}$; max depth $\in \{3, 6\}$; learning rate $\in \{0.05, 0.1\}$
LSTM	1 layer $\times$ 32 units + linear head; Adam, lr 0.01; 60 epochs; batch 32; z-score inputs; seed 42

C. Dataset Characteristics

Table IV summarises the trace and Fig. 4 plots it. Within each day the trace is a two-level step with noise; across days it follows the fixture schedule, rising through the semi-finals (15–16 Nov) to the Final (19 Nov) and collapsing after the tournament (20–25 Nov, hours 1200–1343).

TABLE IV
THE 2023 CRICKET WORLD CUP HOURLY TRACE

Property	Value
Source	Wikimedia Pageviews, en.wikipedia, "2023 Cricket World Cup", agent = user [39]
Span	1 Oct – 25 Nov 2023; 56 days; 1,344 hourly intervals
Total workload	29,957,762 requests (equal to the sum of the daily counts)
Deployment window	Hours 264–1343 (12 Oct – 25 Nov): 25,636,322 requests; mean 23,737 per hour
Peak	117,612 requests at 17:00, 19 Nov 2023 (the Final; hour 1193)
Hourly profile	8.75% of daily traffic per match hour (14:00–21:00), 1.875% per other hour; noise $\mathcal{U}(0.92, 1.08)$
Peak fleet	30 VMs required; 33 with headroom

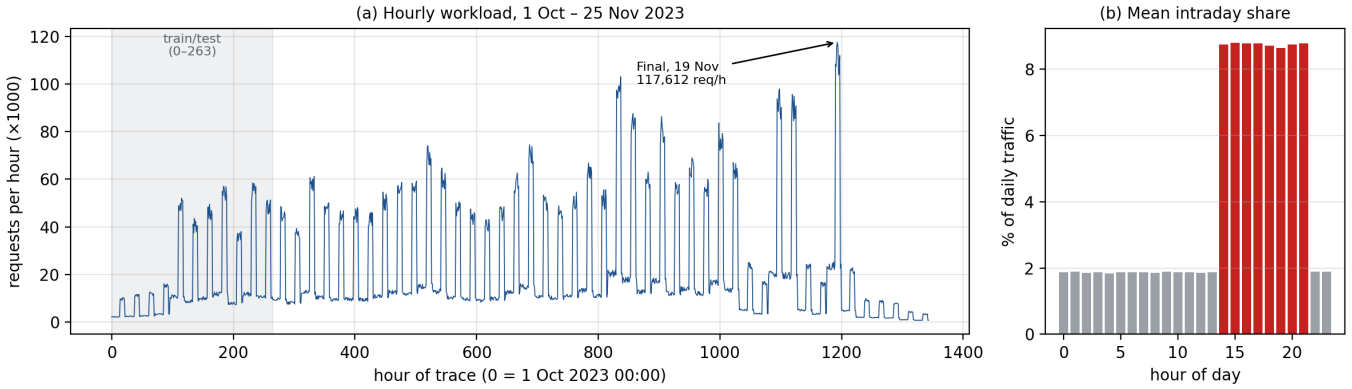


Fig. 4. (a) Hourly cricket workload, 1 Oct – 25 Nov 2023; the shaded region is the initial training and test window. (b) Mean share of daily traffic by hour of day, showing the 70/30 match-hour step profile.

D. Prediction Accuracy

The initial test window (hours 240–263) ranked Random Forest first (Score 6.49%), ahead of XGBoost (7.07%), SVR (11.75%), LSTM (17.84%) and ARIMA (29.18%). One retraining event occurred, at hour 552, and the re-test

again selected Random Forest (5.42%, ahead of XGBoost 7.15%, SVR 7.48%, LSTM 12.22%, ARIMA 33.22%). Random Forest was therefore the active model for all 1,080 deployment hours (Table V), although, as shown below, the best-scoring model changed from cycle to cycle.

TABLE V
ACTIVE-MODEL TIMELINE AND PER-SEGMENT ACCURACY (SCORE %, EQ. 3)

Segment (hours)	Active model	Mean load (req/h)	MTTD Score	Static RF Score
264–551	RF (initial)	24,382	24.83	24.83
552–1175	RF (retrained)	26,822	24.16	56.83
1176–1199 (Final)	RF (retrained)	52,289	79.11	50.56
1200–1343 (post-tournament)	RF (retrained)	4,320	189.97	248.35
All (264–1343)		23,737	32.78	52.43

Retraining more than halved the error during the tournament (24.16% vs. 56.83% for hours 552–1175) and reduced it after the tournament, but on the day of the Final the retrained model scored worse than the static one (79.11% vs. 50.56%). Table VI traces every threshold breach across the 45 deployment cycles.

TABLE VI
THRESHOLD BREACHES ACROSS THE 45 DEPLOYMENT CYCLES

Cycle	Start hour	Score%	Threshold%	k	Outcome (next cycle)
2	288	10.69	10.51	1	Absorbed ($7.51 \leq 16.04$)
11	504	20.75	6.94	1	Watch
12	528	97.48	31.13	2	Retrain $\rightarrow$ RF
24	816	29.87	11.07	1	Absorbed ($17.21 \leq 44.81$)
27	888	13.65	8.62	1	Absorbed ($4.98 \leq 20.48$)
31	984	10.89	8.57	1	Absorbed ($5.26 \leq 16.33$)
33	1032	90.14	9.20	1	Absorbed ($102.85 \leq 135.21$)
37	1128	90.16	40.53	1	Absorbed ($80.70 \leq 135.24$)
40	1200	166.82	138.45	1	Absorbed ($130.00 \leq 250.23$)
44	1296	386.87	296.31	1	Absorbed ($523.03 \leq 580.30$)

Ten breaches produced one retrain; eight single-cycle breaches were absorbed because the following cycle fell below the threshold set by Eq. (6). Fig. 5 shows the per-cycle scores of all five candidates: the lowest-scoring model changed frequently between Random Forest, SVR, XGBoost and ARIMA, while the active model changes only at a retraining event.

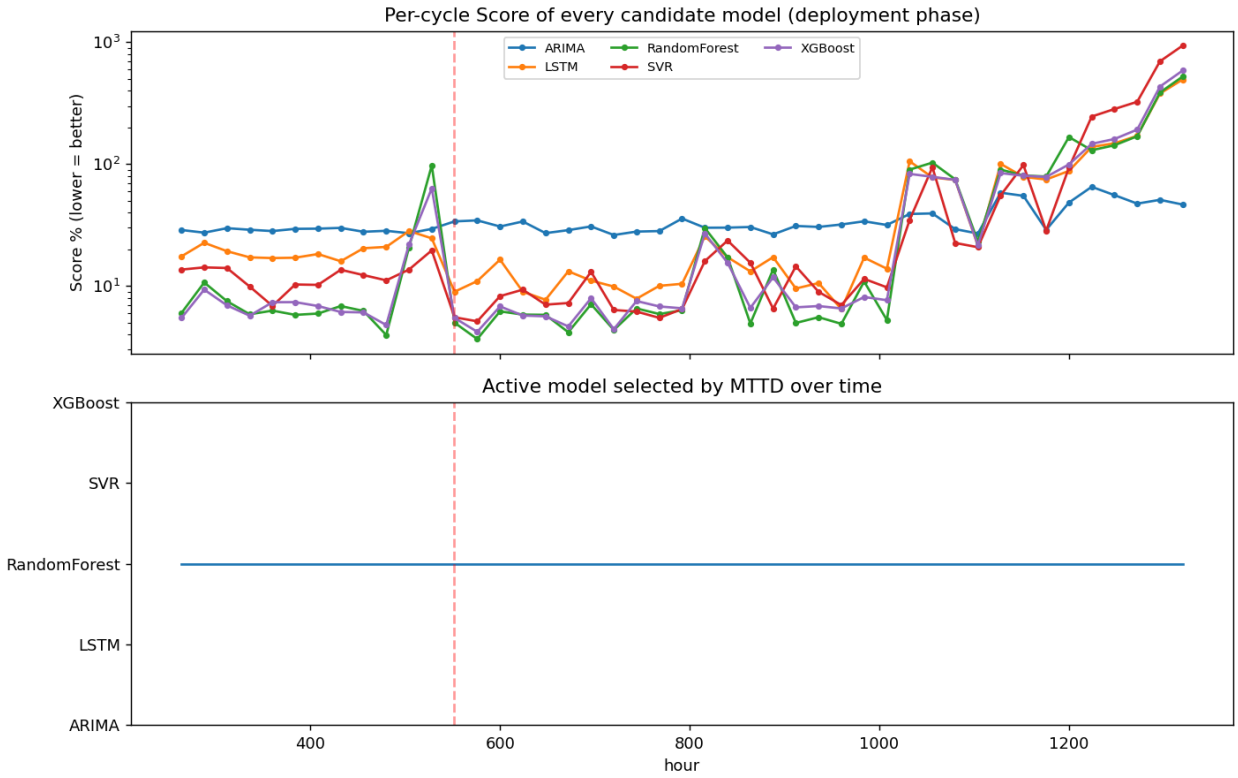


Fig. 5. Top: per-cycle Score of all five candidates (log scale); the lowest line in each cycle is that cycle's best model. Bottom: active model selected by MTTD; the dashed line marks the retraining event at hour 552.

Table VII compares accuracy over all 1,080 deployment hours; the per-model rows are retrained at the same event as MTTD. Against the static predictor, MTTD reduces the score by 37.5%, RMSE by 28.7% and MAE by 47.8%. Because Random Forest was always active, the MTTD row coincides with the retrained Random Forest. The last column counts how often each model had the lowest score in a cycle. Fig. 6 shows the forecasts against the actual load.

TABLE VII
ACCURACY OVER ALL 1,080 DEPLOYMENT HOURS

Predictor	Score%	RMSE	MAE	Best cycles (of 45)
SVR (hindsight)	22.41	–	–	11
ARIMA	30.58	–	–	8
XGBoost	30.66	–	–	10
LSTM	31.75	–	–	0
MTTD (= retrained RF)	**32.78**	**13,649.96**	**5,945.88**	16
Static RF (no retraining)	52.43	19,154.94	11,397.48	–

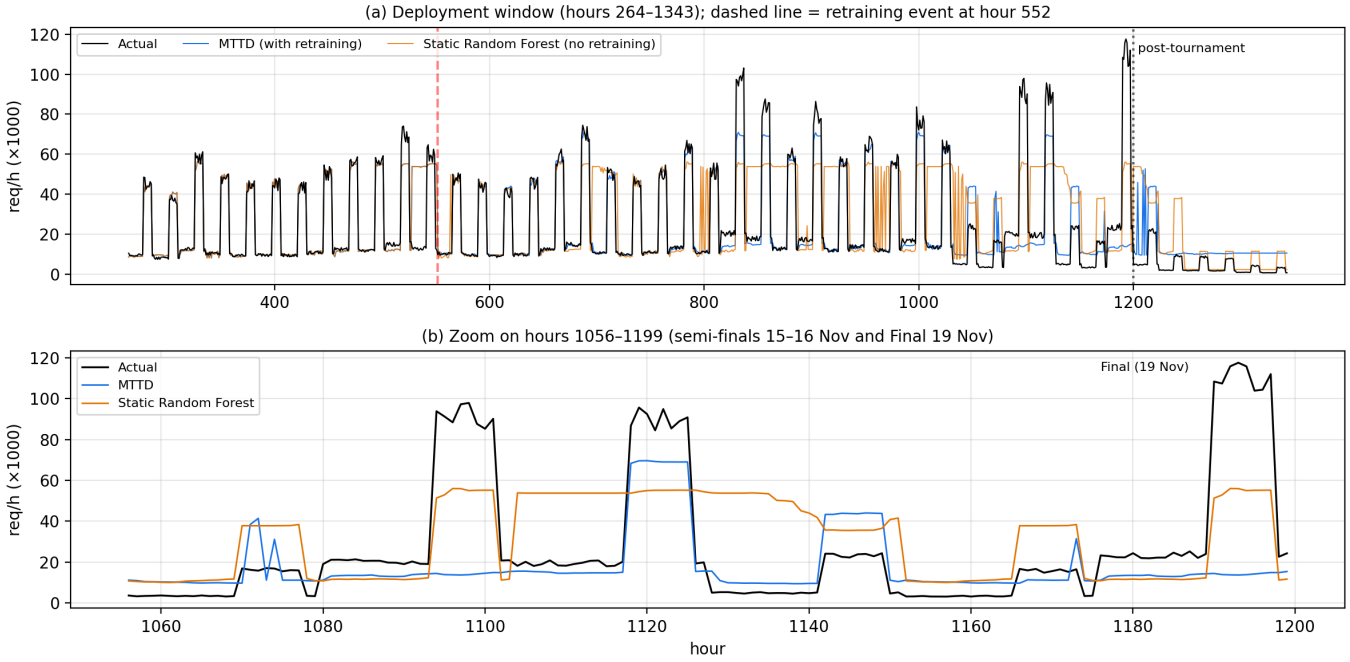


Fig. 6. (a) Actual load, MTTD forecast and static Random Forest forecast over the deployment window. (b) Zoom on hours 1056–1199, covering the semi-finals and the Final.

E. Synthetic Benchmark Validation

We ran each predictor on the four synthetic patterns of [1] (697 minutes, 70/30 split, 100-point test window, window of 5) using `run_benchmark.py`. Table VIII and Fig. 7 report the results.

TABLE VIII
SYNTHETIC BENCHMARKS: SCORE % (MAPE IN PARENTHESES); BEST PER PATTERN IN BOLD

Model	Uniform (bell)	Linear low (0.35)	Linear high (2.31)	Bounded random walk
ARIMA	**1.51 (1.42%)**	**0.49 (0.44%)**	**0.08 (0.07%)**	**3.63 (3.28%)**
SVR	11.11 (11.49%)	50.95 (49.73%)	54.28 (54.05%)	7.73 (7.00%)
Random Forest	3.95 (3.96%)	10.56 (9.61%)	10.25 (9.29%)	3.96 (3.66%)
XGBoost	4.35 (4.31%)	10.46 (9.51%)	10.28 (9.32%)	3.83 (3.55%)
LSTM	3.70 (3.80%)	2.29 (2.17%)	2.14 (2.06%)	3.69 (3.40%)

In our reproduction ARIMA scored best on all four patterns, whereas the original paper reports SVR as the best model on its random workload [1]. On the linear trends, Random Forest, XGBoost and SVR forecasts stay flat or fall below the rising series, while the LSTM tracks the trend with a small offset. On the cricket trace the ranking changes: non-seasonal ARIMA cannot represent the daily step and stays near 30% on most cycles (Fig. 5).

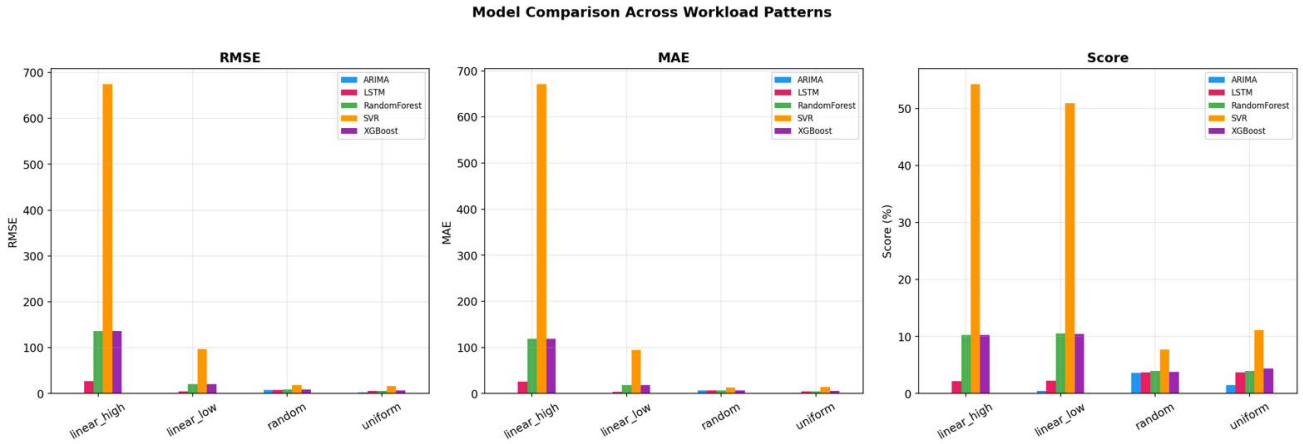


Fig. 7. RMSE, MAE and Score of the five predictors across the four synthetic workload patterns.

F. CloudSim Provisioning Results

Table IX reports the outcomes of the full CloudSim run over 1,080 hours (Fig. 8); required capacity totals 6,981 VM-hours. Fig. 9 plots cost against SLA outcome.

TABLE IX
CLOUDSIM PROVISIONING OUTCOMES OVER 1,080 HOURS

Policy	VM-hours	vs. ORACLE	Wasted VMh	SLA violations	Under-prov. hours	Mean makespan (s)
ORACLE	7,591	–	610	0 (0.00%)	0	2,881.2
REACTIVE	7,593	+0.03%	1,064	56 (5.19%)	56	3,216.8
MTTD-Retrain	7,475	−1.53%	1,077	145 (13.43%)	145	3,036.2
MTTD-Hybrid	8,707	+14.70%	1,796	20 (1.85%)	20	2,585.5
MTTD-noRetrain	9,018	+18.80%	2,623	171 (15.83%)	171	2,639.7

```
$ java -cp build:cloudsim-3.0.3.jar mttd.ProvisioningSimulation   # MAX_HOURS = 0
=== Models used by MTTD (predictions_retrain.csv) ===
  RandomForest      1080 hours (100.0%)
  No model switches

=== Models used by MTTD (predictions_noretrain.csv) ===
  RandomForest      1080 hours (100.0%)
  No model switches

Simulating 1080 hours x 5 policies ...
  ... 1000/1080 hours done

Finished in 10 s

Policy             VM-hours   Wasted VMh    SLA viol. Under-prov hrs Avg resp (s)
REACTIVE               7593         1064      56 ( 5.2%)         56           3217
MTTD-noRetrain         9018         2623     171 (15.8%)        171           2640
MTTD-Retrain           7475         1077     145 (13.4%)        145           3036
ORACLE                 7591          610       0 ( 0.0%)          0           2881
MTTD-Hybrid            8707         1796      20 ( 1.9%)         20           2585

Per-hour details written to cloudsim_results.csv
```

Fig. 8. Console output of the full CloudSim 3.0.3 run (MAX_HOURS = 0, 1,080 hours × 5 policies); intermediate progress lines trimmed.

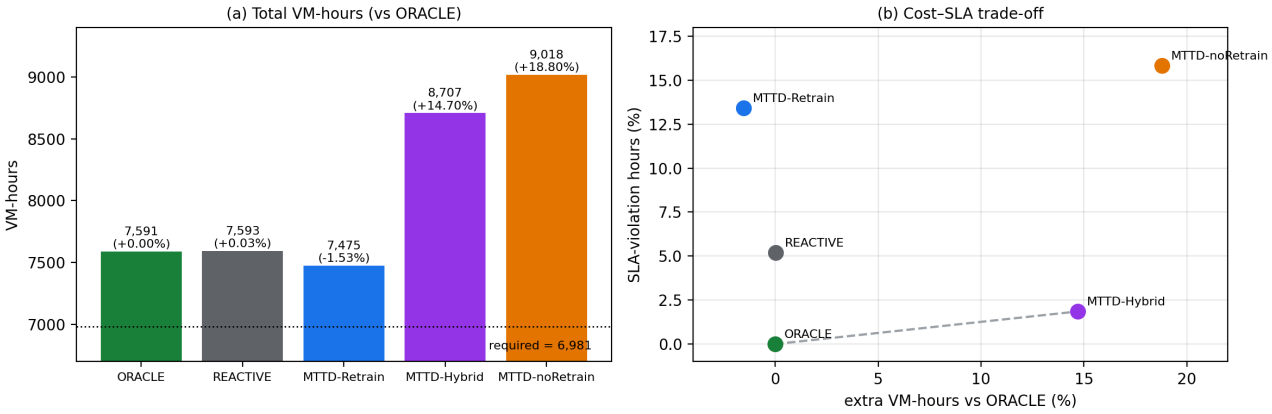


Fig. 9. (a) Total VM-hours per policy relative to ORACLE. (b) Cost–SLA trade-off: VM-hours relative to ORACLE versus SLA-violation rate.

Compared with MTTD-noRetrain, MTTD-Retrain used 17.1% fewer VM-hours (7,475 vs. 9,018), 58.9% fewer wasted VM-hours (1,077 vs. 2,623) and 15.2% fewer SLA-violation hours (145 vs. 171). Its VM-hours were 1.5% below ORACLE because it under-provisioned in 145 hours; REACTIVE violated the SLA in 56 hours (44 of them at the daily 14:00 step-up), and MTTD-Hybrid in 20 hours at 14.7% more VM-hours than ORACLE.

G. *Effect of Reselection Frequency*

In MTTD the active model is re-chosen only when retraining is triggered, which on this trace happened once. To show how model choice evolves with more frequent reselection, we ran two variants using `mttd_bridge.py` unchanged except for when reselection happens (`reselection_study.py`). In the first, the logged forecasts of all five candidates from the standard run were used to re-choose the model every 24 hours or every hour, selecting the model with the lowest Score (Eq. 3) over the previous 6, 24 or 72 hours, with no additional retraining. In the second, retraining and reselection were performed at the end of every 24-hour cycle (44 retraining events). Provisioning outcomes were computed with Eqs. (12)–(14), which reproduce the CloudSim values of Table IX exactly for the standard runs.

TABLE X
EFFECT OF RESELECTION FREQUENCY OVER THE 1,080 DEPLOYMENT HOURS

Strategy	Score%	Switches	Models used	VM-hours	SLA violations
Standard MTTD (two-strike retraining)	32.78	0	1	7,475	145
Reselect every 24 h on last 24 h	18.55	23	4	7,493	138
Reselect every 24 h on last 72 h	16.30	11	4	7,503	139
Reselect every 1 h on last 24 h	15.42	60	5	7,481	127
Reselect every 1 h on last 6 h	19.93	262	5	7,413	120
Retrain + reselect every 24 h	16.13	26	5	7,346	124

More frequent reselection lowered the score from 32.78% to between 15.42% and 19.93%, below the best single model (SVR, 22.41%). With retraining and reselection every day, the active model changed 26 times and all five models were used (Fig. 10): Random Forest and XGBoost were selected most often during the group stage, and SVR from the Final onward. Hourly reselection on the last 24 hours gave the lowest score (15.42%) without additional retraining; shortening the window to 6 hours produced 262 switches and a higher score (19.93%). SLA-violation hours fell from 145 to between 120 and 139, remaining above REACTIVE (56).

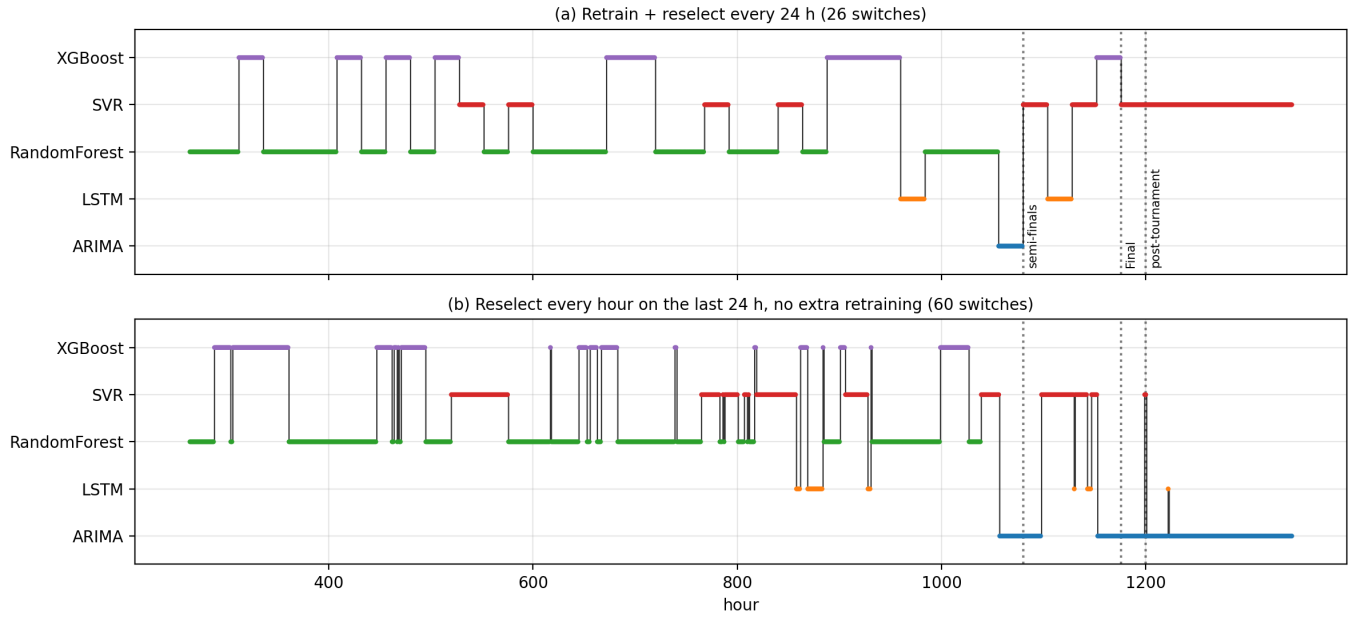


Fig. 10. Active model over the deployment window. (a) Retraining and reselection every 24 hours. (b) Reselection every hour on the last 24 hours using the standard run's forecasts. Dotted lines mark the semi-finals (hour 1080), the Final (1176) and the end of the tournament (1200).

H. Comparison with the Original Study

On the synthetic patterns our reproduction agrees with [1] that ARIMA performs well on smooth uniform and linear workloads, but differs on the random workload, where the original reports SVR as best and our ARIMA scored lowest. On the cricket trace, the pipeline behaved as designed: the test phase selected the lowest-scoring model, isolated bad cycles were absorbed, and two consecutive breaches triggered retraining and reselection. The best-scoring model changed between cycles, yet the active model changes only at retraining, so Random Forest remained active while SVR had the lowest score over the full deployment (22.41%). In the standard run neither added model became the active model, although XGBoost was the best model in 10 cycles; with more frequent reselection (Table X) both XGBoost and the LSTM were selected.

I. Threats to Validity

Semi-synthetic data. Only the daily totals are measured; the hourly shape, match window and noise are modelled, so models that learn the 24-hour step learn a property of the generator. *Proxy traffic.* Page views are not server requests and their burst statistics may differ from HTTP logs such as FIFA 1998 [13]. *Single trace and seed.* Results come from one tournament, one noise seed and one train/test configuration. *Cross-validation.* `GridSearchCV` with ordinary k-fold validation does not respect time order. *Non-seasonal ARIMA.* ARIMA was run without a seasonal term. *Simulation simplifications.* The simulation ignores VM boot time and request-level latency and uses hour-long decision intervals.

V. Conclusion and Future Work

We reproduced the MTTD framework of Nashaat *et al.* with five candidate models instead of three and a 2023 Cricket World Cup workload instead of FIFA logs, and replayed its forecasts in CloudSim. On the synthetic patterns ARIMA scored best. On the cricket trace the test phase selected Random Forest, the best-scoring model varied from cycle to cycle, and one retraining event occurred; retraining reduced the deployment score from 52.43% to 32.78% and, in CloudSim, VM-hours by 17.1% and SLA-violation hours by 15.2% compared with no retraining. Re-selecting the model every day or every hour made the active model change across all five candidates and lowered the score to as little as 15.42%.

Future work includes validating on measured hourly or minute-level server logs of live sports events, reducing SLA violations during sudden peaks, integrating MTTD forecasts with the Kubernetes Horizontal Pod Autoscaler [34], and using multivariate inputs such as CPU, memory and network metrics.

REFERENCES

[1] M. Nashaat *et al.*, "A dynamic machine learning approach for workload prediction in cloud environments," *Sci. Rep.*, vol. 16, art. no. 10983, 2026, doi: 10.1038/s41598-026-40777-z.

[2] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose, and R. Buyya, "CloudSim: A toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," *Softw. Pract. Exper.*, vol. 41, no. 1, pp. 23–50, 2011.

[3] R. N. Calheiros, E. Masoumi, R. Ranjan, and R. Buyya, "Workload prediction using ARIMA model and its impact on cloud applications' QoS," *IEEE Trans. Cloud Comput.*, vol. 3, no. 4, pp. 449–458, 2015.

[4] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.

[5] R. J. Hyndman and Y. Khandakar, "Automatic time series forecasting: The forecast package for R," *J. Stat. Softw.*, vol. 27, no. 3, pp. 1–22, 2008.

[6] H. Drucker, C. J. C. Burges, L. Kaufman, A. Smola, and V. Vapnik, "Support vector regression machines," in *Proc. NIPS*, 1997, pp. 155–161.

[7] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794.

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[10] M. E. Karim, M. M. S. Maswood, S. Das, and A. G. Alharbi, "BHyPreC: A novel Bi-LSTM based hybrid recurrent neural network model to predict the CPU workload of cloud virtual machine," *IEEE Access*, vol. 9, pp. 131476–131495, 2021.

[11] T. Lorido-Botran, J. Miguel-Alonso, and J. A. Lozano, "A review of auto-scaling techniques for elastic applications in cloud environments," *J. Grid Comput.*, vol. 12, no. 4, pp. 559–592, 2014.

[12] C. Qu, R. N. Calheiros, and R. Buyya, "Auto-scaling web applications in clouds: A taxonomy and survey," *ACM Comput. Surv.*, vol. 51, no. 4, art. no. 73, 2018.

[13] M. Arlitt and T. Jin, "A workload characterization study of the 1998 World Cup Web site," *IEEE Netw.*, vol. 14, no. 3, pp. 30–37, 2000.

[14] J. Jung, B. Krishnamurthy, and M. Rabinovich, "Flash crowds and denial of service attacks: Characterization and implications for CDNs and web sites," in *Proc. 11th Int. World Wide Web Conf.*, 2002, pp. 293–304.

[15] N. Roy, A. Dubey, and A. Gokhale, "Efficient autoscaling in the cloud using predictive models for workload forecasting," in *Proc. IEEE 4th Int. Conf. Cloud Comput.*, 2011, pp. 500–507.

[16] S. Islam, J. Keung, K. Lee, and A. Liu, "Empirical prediction models for adaptive resource provisioning in the cloud," *Future Gener. Comput. Syst.*, vol. 28, no. 1, pp. 155–162, 2012.

[17] Z. Gong, X. Gu, and J. Wilkes, "PRESS: PRedictive Elastic reSource Scaling for cloud systems," in *Proc. Int. Conf. Netw. Serv. Manage. (CNSM)*, 2010, pp. 9–16.

[18] Z. Shen, S. Subbiah, X. Gu, and J. Wilkes, "CloudScale: Elastic resource scaling for multi-tenant cloud systems," in *Proc. 2nd ACM Symp. Cloud Comput.*, 2011, art. no. 5.

[19] I. K. Kim, W. Wang, Y. Qi, and M. Humphrey, "Empirical evaluation of workload forecasting techniques for predictive cloud resource scaling," in *Proc. IEEE 9th Int. Conf. Cloud Comput.*, 2016, pp. 1–10.

[20] I. K. Kim, W. Wang, Y. Qi, and M. Humphrey, "Forecasting cloud application workloads with CloudInsight for predictive resource management," *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 1848–1863, 2022.

[21] J. Kumar and A. K. Singh, "Workload prediction in cloud using artificial neural network and adaptive differential evolution," *Future Gener. Comput. Syst.*, vol. 81, pp. 41–52, 2018.

[22] J. Kumar, A. K. Singh, and R. Buyya, "Self directed learning based workload forecasting model for cloud resource management," *Inf. Sci.*, vol. 543, pp. 345–366, 2021.

[23] J. Gao, H. Wang, and H. Shen, "Machine learning based workload prediction in cloud computing," in *Proc. 29th Int. Conf. Comput. Commun. Netw. (ICCCN)*, 2020, pp. 1–9.

[24] J. Bi, S. Li, H. Yuan, and M. Zhou, "Integrated deep learning method for workload and resource prediction in cloud systems," *Neurocomputing*, vol. 424, pp. 35–48, 2021.

[25] C. Liu, C. Liu, Y. Shang, S. Chen, B. Cheng, and J. Chen, "An adaptive prediction approach based on workload pattern discrimination in the cloud," *J. Netw. Comput. Appl.*, vol. 80, pp. 35–44, 2017.

[26] K. Cetinski and M. B. Juric, "AME-WPC: Advanced model for efficient workload prediction in the cloud," *J. Netw. Comput. Appl.*, vol. 55, pp. 191–201, 2015.

[27] V. R. Messias, J. C. Estrella, R. Ehlers, M. J. Santana, R. C. Santana, and S. Reiff-Marganiec, "Combining time series prediction models using genetic algorithm to autoscale Web applications hosted in the cloud infrastructure," *Neural Comput. Appl.*, vol. 27, no. 8, pp. 2383–2406, 2016.

[28] M. Amiri and L. Mohammad-Khanli, "Survey on prediction models of applications for resources provisioning in cloud," *J. Netw. Comput. Appl.*, vol. 82, pp. 93–113, 2017.

[29] M. Masdari and A. Khoshnevis, "A survey and classification of the workload forecasting methods in cloud computing," *Cluster Comput.*, vol. 23, no. 4, pp. 2399–2424, 2020.

[30] R. Moreno-Vozmediano, R. S. Montero, E. Huedo, and I. M. Llorente, "Efficient resource provisioning for elastic cloud services based on machine learning techniques," *J. Cloud Comput.*, vol. 8, art. no. 5, 2019.
[31] N. R. Herbst, S. Kounev, and R. Reussner, "Elasticity in cloud computing: What it is, and what it is not," in *Proc. 10th Int. Conf. Autonomic Comput. (ICAC)*, 2013, pp. 23–27.
[32] K. Rzadca *et al.*, "Autopilot: Workload autoscaling at Google," in *Proc. 15th Eur. Conf. Comput. Syst. (EuroSys)*, 2020, art. no. 16.
[33] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at Google with Borg," in *Proc. 10th Eur. Conf. Comput. Syst. (EuroSys)*, 2015, art. no. 18.
[34] The Kubernetes Authors, "Horizontal Pod Autoscaling," Kubernetes Documentation. [Online]. Available: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/
[35] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, art. no. 44, 2014.
[36] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *Int. J. Forecast.*, vol. 22, no. 4, pp. 679–688, 2006.
[37] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
[38] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 8024–8035.
[39] Wikimedia Foundation, "Wikimedia REST API: Pageviews per-article endpoint." [Online]. Available: https://wikimedia.org/api/rest_v1/